LEARNING GUIDE · MINI PROJECT AI

Knight Trap

Implementasi AI pada Game Knight Trap Menggunakan Algoritma Minimax dengan Alpha-Beta Pruning

Louis Yuriel Saputra 0706012410050

Bagianmu: Minimax, Alpha-Beta Pruning, tingkat kesulitan, panel statistik, uji kebenaran Alpha-Beta

Artificial Intelligence · Information Management & Technology · Universitas Ciputra 2026/2027
Disusun dari proposal (v1.1) dan kode di repo KnightTrap · 29 Oktober 2026

Daftar Isi

1. Peta Belajarmu
2. Gambaran Proyek (Proposal Bab 1)
3. Aturan Game → Kode (Proposal Bab 2.1)
4. Alur Satu Giliran di Kode
5. Konsep AI yang Wajib Dipahami Semua Anggota
6. Minimax: Dari Pseudocode ke Kode
7. Alpha-Beta Pruning
8. Langkah, make/unmake, dan Kandidat Jebakan
9. Tiga Pembatasan Agar Cepat
10. Tingkat Kesulitan, Tombol Alpha-Beta, Panel Statistik
11. Pengujian Kebenaran Alpha-Beta
12. Hasil Pengujian (Proposal Bab 2.2)
13. Fitur Rally: Poin, Leaderboard, Admin
14. Latihan
15. Pertanyaan Presentasi yang Mungkin Muncul
16. Checklist Sebelum Presentasi

1 Peta Belajarmu

Setelah mempelajari guide ini, kamu bisa:

- Menjelaskan Minimax dan Alpha-Beta Pruning dengan contoh pohon kecil, termasuk kapan cutoff terjadi.
- Menunjuk baris kode `search()` di `js/ai.js` untuk setiap baris pseudocode Gambar 2.1.1.
- Menjelaskan tiga pembatasan: kandidat jebakan maks. 12, pengurutan langkah, dan batas waktu 4 detik.
- Menjelaskan tingkat kesulitan, tombol Alpha-Beta ON/OFF, dan isi panel statistik "Otak AI".
- Membaca dan membela hasil uji kebenaran Alpha-Beta (30 posisi, depth 1–3).

Tugasmu di Timeline (Bab 2.3)

Minggu	Pekerjaan	Kode
2	Minimax dasar: pohon permainan, batas kedalaman, pilih langkah terbaik	`ai.js`: `search()`, `ab()`, `generateMoves()`, `make()`/`unmake()`
2–3	Alpha-Beta, pengurutan langkah, kandidat jebakan, batas 4 detik, tombol pemangkasan	`ai.js`: `orderMoves()`, `trapCandidates()`, `ABORT`; `main.js`: tombol `#game-ab`
3	Tingkat kesulitan dan panel statistik AI	`config.js`: `DIFFICULTY`; `main.js`: `startAiGame()`, `ui.stats()`
3	Pengujian Minimax vs Alpha-Beta	`tests/run.mjs` bagian 2

Urutan belajar yang disarankan: Bab 2–5 (dasar bersama) → Minimax → Alpha-Beta → Langkah & make/unmake → Tiga pembatasan → Level & panel → Pengujian → Latihan → Pertanyaan presentasi.

2 Gambaran Proyek (Proposal Bab 1)

Apa yang kita buat

Knight Trap adalah varian dari permainan papan *Isolation* (1972). Dua knight ada di papan 7 × 7. Pada gilirannya, pemain:

1. **Lompat**: menggerakkan knight seperti kuda catur (huruf L) ke kotak terbuka.
2. **Bakar**: kotak yang ditinggalkan langsung terbakar dan tidak bisa dipakai lagi.
3. **Jebak**: menghancurkan satu kotak terbuka lain di mana pun.

Pemain yang **tidak punya lompatan legal** pada gilirannya kalah. Game ini dibuat untuk **rally games** (NPLC Universitas Ciputra, tema *Human vs AI*), jadi aturannya harus bisa dijelaskan dalam satu menit, satu game selesai dalam beberapa menit, dan lawan AI-nya tidak boleh terlalu mudah.

Masalah yang diselesaikan (Bab 1.2)

Kita harus membuat lawan AI yang (1) memilih langkah yang kuat, (2) cukup cepat untuk browser, dan (3) punya beberapa tingkat kesulitan. Ada dua tantangan utama:

- **Pohon permainan besar.** Satu langkah = pilihan lompatan (maks. 8) × pilihan kotak jebakan (bisa puluhan). Jumlah posisi meledak setiap giliran.
- **Menilai posisi tanpa main sampai habis.** Kita butuh *fungsi evaluasi* yang memberi skor pada posisi di tengah permainan.

Solusinya, sesuai ketentuan tugas yang membatasi kita ke **empat algoritma**: **Minimax** + **Alpha-Beta Pruning** untuk mengambil keputusan, dan **BFS** + **DFS** sebagai dasar fungsi evaluasi.

Peta file kode

File	Isi	Pemilik utama (Timeline)
`js/config.js`	Konstanta: ukuran papan, posisi awal, kedalaman tiap level, batas AI, poin	Shatrya (setup), Louis (AI)
`js/rules.js`	Aturan game: lompatan legal, bakar, jebakan, cek kalah. Tidak menyentuh DOM	Shatrya
`js/ai.js`	Minimax + Alpha-Beta, BFS, DFS, fungsi evaluasi, pemain acak & greedy	Louis (search), Clarrence (BFS), Shatrya (DFS)
`js/ai.worker.js`, `js/ai-client.js`	Menjalankan AI di Web Worker supaya tampilan tidak macet	Louis
`js/game.js`	Alur giliran: input pemain, giliran AI, animasi, game over	Shatrya & Louis
`js/board.js`, `js/fx.js`, `css/style.css`, `index.html`	Tampilan papan, knight, efek api/jebakan, layout	Clarrence
`js/main.js`, `js/i18n.js`, `js/audio.js`	Layar menu, pengaturan, panel statistik, teks dua bahasa, suara	Clarrence & Louis
`js/net.js`	Multiplayer online (PeerJS)	Tambahan di luar proposal
`js/scoring.js`, `api/*`, `admin.html`	Poin, leaderboard, halaman admin (fitur rally)	Tambahan di luar proposal
`tests/run.mjs`	Semua pengujian Bab 2.2	Louis, Clarrence (pengujian)

> **Repo dengan komentar per baris.** Folder `KnightTrap-Annotated` berisi kode yang sama persis, tetapi setiap baris diberi komentar bahasa Indonesia. Pakai repo itu untuk belajar; potongan kode di guide ini diambil dari repo asli `KnightTrap` supaya nomor barisnya cocok dengan kode yang dijalankan.

3 Aturan Game → Kode (Proposal Bab 2.1)

Setiap aturan di proposal punya tempat yang jelas di `js/rules.js`. File ini adalah **satu-satunya sumber aturan**: tampilan, AI, mode online, dan server leaderboard semuanya memakai fungsi yang sama.

Aturan proposal	Kode
Papan 7 × 7, array 49 kotak (kosong / terbakar / jebakan) + posisi 2 knight	`createState()`, konstanta `EMPTY`, `BURNED`, `TRAP`
(a) Knight bergerak huruf L	`JUMPS` dan `NEIGHBORS` (dihitung sekali di awal)
(b) Kotak yang ditinggalkan terbakar	`applyJump()`
(c) Hancurkan satu kotak terbuka	`applyTrap()`; validasi di `validateMove()`
(d) Tidak boleh mendarat di kotak terbakar, jebakan, atau kotak lawan	`isOpen()` dipakai oleh `legalJumps()`
(e) Tidak punya lompatan legal = kalah	`winner()` memakai `canMove()`

Representasi papan

Kotak diberi nomor 0 sampai 48: `index = baris × 7 + kolom`, baris 0 adalah baris paling atas (rank 7). Jadi **a1 = 42** (kiri bawah) dan **g7 = 6** (kanan atas).

js/config.js : baris 3–8

```
3  export const N = 7;
4  export const SIZE = N * N;
5
6  // The proposal does not fix start squares. P1 starts at a1 (bottom-left),
7  // P2 at g7 (top-right). Index = row * 7 + col, row 0 is the top rank (7).
8  export const START = Object.freeze({ p1: 42, p2: 6, first: 0 });
```

js/rules.js : baris 28–35

```
28 export function createState(start = START) {
29   return {
30     cells: new Int8Array(SIZE),
31     pos: [start.p1, start.p2],
32     turn: start.first ?? 0,
33     moves: 0,
34   };
35 }
```

`cells` memakai `Int8Array` (array angka 8-bit) karena kecil dan cepat disalin. `pos[0]` adalah posisi knight P1, `pos[1]` knight P2, `turn` adalah siapa yang jalan (0 atau 1).

Tetangga knight dihitung sekali

js/rules.js : baris 10–26

```
const JUMPS = [
  [-2, -1], [-2, 1], [-1, -2], [-1, 2],
  [1, -2], [1, 2], [2, -1], [2, 1],
];

// Knight neighbor lists, computed once.
export const NEIGHBORS = Array.from({ length: SIZE }, (_, i) => {
  const r = Math.floor(i / N);
  const c = i % N;
  const out = [];
  for (const [dr, dc] of JUMPS) {
    const rr = r + dr;
    const cc = c + dc;
    if (rr >= 0 && rr < N && cc >= 0 && cc < N) out.push(rr * N + cc);
  }
  return out;
});
```

Untuk setiap kotak, kita coba 8 arah L dan simpan hanya yang masih di dalam papan. Hasilnya tabel 49 daftar. AI memanggil daftar ini jutaan kali, jadi menghitungnya sekali di awal menghemat banyak waktu.

Satu giliran = lompat, bakar, jebak

js/rules.js : baris 53–55

```
export function isOpen(s, sq) {
  return s.cells[sq] === EMPTY && sq !== s.pos[0] && sq !== s.pos[1];
}
```

js/rules.js : baris 57–61

```
export function legalJumps(s, side = s.turn) {
  const out = [];
  for (const n of NEIGHBORS[s.pos[side]]) if (isOpen(s, n)) out.push(n);
  return out;
}
```

js/rules.js : baris 74–80

```
export function applyJump(s, to) {
  const side = s.turn;
  const from = s.pos[side];
  s.cells[from] = BURNED;
  s.pos[side] = to;
  return from;
}
```

js/rules.js : baris 83–85

```
export function applyTrap(s, sq) {
  if (sq >= 0) s.cells[sq] = TRAP;
}
```

js/rules.js : baris 111–113

```
111  export function winner(s) {
112    return canMove(s, s.turn) ? null : s.turn ^ 1;
113  }
```

Kenapa `winner()` mengembalikan `s.turn ^ 1`? `^` adalah XOR. `0 ^ 1 = 1` dan `1 ^ 1 = 0`, jadi ini cara singkat menulis "pemain lainnya". Kalau yang sedang jalan tidak bisa bergerak, lawannya menang.

4 Alur Satu Giliran di Kode

Bagian ini menghubungkan semua file. Setiap anggota harus bisa menceritakan alur ini saat presentasi.



Gambar: alur satu ronde (pemain lalu AI). Biru = input pemain, krem = alur game, oranye = AI.

js/game.js : baris 119–136

```
119   nextTurn() {
120     if (this.disposed) return;
121     const w = winner(this.state);
122     if (w !== null) return this.finish(w);
123     const p = this.current;
124     if (p.kind === 'human') {
125       this.phase = 'jump';
126       if (this.mode === 'local' && this.history.length) sfx.turn();
127       this.refresh();
128     } else if (p.kind === 'ai') {
129       this.phase = 'ai';
130       this.refresh();
131       this.runAi();
132     } else {
133       this.phase = 'remote';
134       this.refresh();
135     }
136   }
```

js/game.js : baris 138–162

```
  async runAi() {
    this.ui.thinking(true);
    const t0 = performance.now();
    let res = null;
    try {
      res = await think(this.state, { depth: this.ai.depth, prune: this.ai.prune });
    } catch (err) {
      console.error('AI failed', err);
    }
    if (this.disposed) return;
    const elapsed = performance.now() - t0;
    if (elapsed < AICFG.minUiDelayMs) await sleep(AICFG.minUiDelayMs - elapsed);
    if (this.disposed) return;
    this.ui.thinking(false);
    let move = res?.move;
    if (!move || !validateMove(this.state, move)) {
      console.warn('AI returned no valid move; falling back to a random legal move');
      move = randomMove(this.state);
    } else {
      this.ui.stats(res);
    }
    await this.playMove(move);
    if (this.disposed) return;
    this.nextTurn();
  }
```

Perhatikan dua pengaman di `runAi`: (1) kalau AI terlalu cepat (Easy), tampilan menunggu minimal 450 ms supaya tidak terasa "langsung"; (2) kalau AI gagal atau mengembalikan langkah tidak sah, game memakai langkah acak yang legal, jadi game tidak pernah macet.

js/ai-client.js : baris 44–53

```
export function think(state, { depth, prune }) {
  const args = { state: serialize(state), depth, prune };
  const w = getWorker();
  if (!w) return runInline(args);
  return new Promise((resolve, reject) => {
    const id = ++seq;
    pending.set(id, { resolve, reject, args });
    w.postMessage({ id, ...args });
  });
}
```

js/ai.worker.js : baris 5–13

```
self.onmessage = (e) => {
  const { id, state, depth, prune } = e.data;
  try {
    const result = search(deserialize(state), { depth, prune });
    self.postMessage({ id, result });
  } catch (err) {
    self.postMessage({ id, error: String(err && err.message ? err.message : err) });
  }
};
```

Kenapa Web Worker? JavaScript di browser berjalan di satu thread. Kalau Expert berpikir 1–2 detik di thread utama, animasi dan tombol akan membeku. Worker menjalankan `search()` di thread terpisah, jadi papan tetap hidup saat AI berpikir.

5 Konsep AI yang Wajib Dipahami Semua Anggota

Dosen bisa bertanya ke siapa saja. Bagian ini adalah dasar yang harus dikuasai semua anggota, walaupun detailnya milik anggota lain.

Agen dan model P.E.A.S (Tabel 2.1.1)

Komponen	Di Knight Trap	Di kode
Performance	Win rate AI, waktu berpikir ≤ 4 detik, jumlah posisi diperiksa sedikit, tantangan pas untuk siswa SMA	`tests/run.mjs`, panel statistik
Environment	Papan 7 × 7, dua agen bergiliran. Deterministik, fully observable, diskret, berurutan, zero-sum	`createState()`
Actuators	Lompat ke kotak legal (maks. 8 arah) + hancurkan satu kotak	`applyMove()`, `{ jump, trap }`
Sensors	Posisi kedua knight, status 49 kotak, giliran	`state.cells`, `state.pos`, `state.turn`

Kenapa sifat lingkungan penting? Minimax hanya cocok untuk game *deterministik* (tidak ada dadu), *fully observable* (tidak ada kartu tertutup), dan *zero-sum* (untung saya = rugi lawan). Knight Trap memenuhi ketiganya.

Adversarial search dan game tree

Game tree: akar = posisi sekarang, cabang = langkah, simpul = posisi hasil langkah. **Kedalaman** (depth) = berapa giliran ke depan yang dilihat AI. **Branching factor** = rata-rata jumlah langkah per posisi. Di Knight Trap: sampai 8 lompatan × sampai 12 kandidat jebakan = sampai **96 langkah** per posisi. Dengan depth 4: $96^4 \approx 85$ juta posisi. Itulah alasan kita butuh Alpha-Beta dan pembatasan kandidat.

Minimax dalam satu paragraf

AI adalah **MAX** (ingin skor setinggi mungkin), manusia adalah **MIN** (dianggap selalu memilih langkah terburuk untuk AI). AI menelusuri pohon sampai kedalaman tertentu, menilai posisi di ujung dengan fungsi evaluasi, lalu menaikkan nilai ke atas: di simpul MAX ambil nilai terbesar, di simpul MIN ambil nilai terkecil. Langkah di akar dengan nilai terbesar dipilih.

Alpha-Beta dalam satu paragraf

Alpha = nilai terbaik yang sudah pasti didapat MAX. **Beta** = nilai terbaik yang sudah pasti didapat MIN. Kalau di suatu simpul `beta ≤ alpha`, sisa langkah di simpul itu tidak mungkin mengubah keputusan, jadi dilewati (**cutoff**). Hasilnya **sama persis** dengan Minimax biasa, tapi posisi yang diperiksa jauh lebih sedikit. Ini dibuktikan di pengujian.

BFS dan DFS dalam satu paragraf

BFS (antrian, menyebar per "lapisan") menghitung jumlah lompatan minimum dari knight ke setiap kotak. Kotak yang lebih cepat dicapai knight A adalah **wilayah** A. **DFS** (tumpukan, masuk sedalam mungkin dulu)

melakukan **flood fill**: menandai semua kotak yang bisa dicapai knight. Kalau area kedua knight tidak bersentuhan, papan sudah **terbelah** dan pemenangnya hampir pasti yang areanya lebih luas.

js/ai.js : baris 78–108

```
78  export function evaluate(s, me) {
79    const { cells, pos, turn } = s;
80    const areaA = dfs(cells, pos, 0, markA);
81    const areaB = dfs(cells, pos, 1, markB);
82    let split = true;
83    for (let i = 0; i < SIZE; i++) {
84      if (markA[i] === 1 && markB[i] === 1) {
85        split = false;
86        break;
87      }
88    }
89    let score;
90    if (split) {
91      score = (areaA - areaB) * SPLIT_WEIGHT;
92    } else {
93      bfs(cells, pos, 0, distA);
94      bfs(cells, pos, 1, distB);
95      let terr = 0;
96      for (let i = 0; i < SIZE; i++) {
97        if (!open(cells, pos, i)) continue;
98        const a = distA[i];
99        const b = distB[i];
100       if (a < 0 && b < 0) continue;
101       if (b < 0 || (a >= 0 && a < b)) terr++;
102       else if (a < 0 || b < a) terr--;
103       else terr += turn === 0 ? 1 : -1; // tie goes to the side to move
104     }
105     score = terr + 0.5 * (mobility(cells, pos, 0) - mobility(cells, pos, 1));
106   }
107   return me === 0 ? score : -score;
108 }
```

Inti fungsi evaluasi (skor dari sudut pandang P1, lalu dibalik tandanya untuk P2):

- Kalau papan sudah terbelah (DFS): `skor = (areaP1 − areaP2) × 100`.
- Kalau belum: `skor = selisih wilayah (BFS) + 0,5 × selisih mobilitas`. Mobilitas = jumlah lompatan legal saat ini.

Angka 100 (`SPLIT_WEIGHT`) dipilih karena skor wilayah paling besar sekitar 53, jadi papan yang sudah terbelah selalu dinilai lebih "pasti" daripada posisi yang masih terbuka.

6 Minimax: Dari Pseudocode ke Kode

Ide dasarnya

AI tidak tahu langkah mana yang akan dipilih manusia, jadi AI menganggap manusia **selalu memilih langkah terbaik untuk dirinya sendiri** (paling buruk untuk AI). Dengan anggapan itu, AI memilih langkah yang **nilai terburuknya paling baik**. Itulah "min" dan "max".

> **Contoh dengan tangan**
>
> AI (MAX) punya dua langkah, A dan B. Setelah setiap langkah, manusia (MIN) punya dua balasan. Skor di ujung dari fungsi evaluasi:
>
> - Langkah A → balasan menghasilkan skor 3 atau 5. MIN memilih **3**.
> - Langkah B → balasan menghasilkan skor 2 atau 9. MIN memilih **2**.
>
> MAX membandingkan 3 dan 2, lalu memilih **A** (nilai 3). Walaupun B punya skor 9, manusia yang pintar tidak akan memberikannya.

Pemetaan pseudocode Gambar 2.1.1 → kode

Pseudocode	`ab()` di `js/ai.js`
`if player has no legal jump then return terminal score`	`if (!hasJump(s, side)) return side === aiSide ? -(WIN - ply) : WIN - ply;`
`if depth = 0 then return EVALUATE(state)`	`if (d === 0) return evaluate(s, aiSide);`
`moves ← GENERATE(state, player)`	`const moves = generateMoves(s, side);`
`moves ← ORDER(moves)`	`if (d >= CFG.orderMinDepth) orderMoves(...)`
`if player = AI then … best ← max(best, v); alpha ← max(alpha, best)`	blok `if (isMax) { … }`
`else … best ← min(best, v); beta ← min(beta, best)`	blok setelahnya (MIN)
`if beta ≤ alpha then break`	`if (prune && beta <= alpha) { cutoffs++; break; }`
`APPLY(state, move)`	`make(s, m)` lalu `unmake(s, m, from)` setelah rekursi

`js/ai.js : baris 205–294`

```
205  export function search(state, opts = {}) {
206    const depth = Math.max(1, opts.depth ?? 2);
207    const prune = opts.prune ?? true;
208    const timeLimitMs = opts.timeLimitMs ?? CFG.timeLimitMs;
209    const now = opts.now ?? defaultNow;
210    const s = cloneState(state);
211    const aiSide = s.turn;
212    const t0 = now();
213    const deadline = t0 + timeLimitMs;
214    let nodes = 1;
```

```
  let cutoffs = 0;

  function ab(d, alpha, beta, ply) {
    nodes++;
    if ((nodes & 255) === 0 && now() > deadline) throw ABORT;
    const side = s.turn;
    if (!hasJump(s, side)) return side === aiSide ? -(WIN - ply) : WIN - ply;
    if (d === 0) return evaluate(s, aiSide);
    const moves = generateMoves(s, side);
    const isMax = side === aiSide;
    if (d >= CFG.orderMinDepth) orderMoves(s, moves, aiSide, isMax);
    if (isMax) {
      let best = -Infinity;
      for (const m of moves) {
        const from = make(s, m);
        const v = ab(d - 1, alpha, beta, ply + 1);
        unmake(s, m, from);
        if (v > best) best = v;
        if (best > alpha) alpha = best;
        if (prune && beta <= alpha) {
          cutoffs++;
          break;
        }
      }
      return best;
    }
    let best = Infinity;
    for (const m of moves) {
      const from = make(s, m);
      const v = ab(d - 1, alpha, beta, ply + 1);
      unmake(s, m, from);
      if (v < best) best = v;
      if (best < beta) beta = best;
      if (prune && beta <= alpha) {
        cutoffs++;
        break;
      }
    }
    return best;
  }

  if (!hasJump(s, aiSide)) return null;
  const moves = generateMoves(s, aiSide);
  if (depth >= CFG.orderMinDepth) orderMoves(s, moves, aiSide, true);

  let best = -Infinity;
  let bestMove = moves[0];
  let alpha = -Infinity;
  let completed = 0;
  let timedOut = false;
  try {
    for (const m of moves) {
      const from = make(s, m);
      const v = ab(depth - 1, alpha, Infinity, 1);
      unmake(s, m, from);
      completed++;
      if (v > best) {
        best = v;
        bestMove = m;
      }
      if (prune && best > alpha) alpha = best;
    }
  } catch (e) {
```

```
    if (e !== ABORT) throw e;
    timedOut = true;
  }

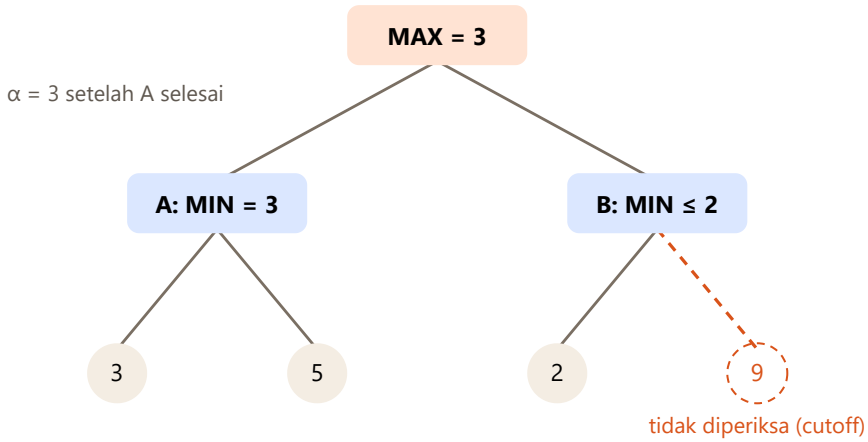
  return {
    move: unpack(bestMove),
    score: completed > 0 ? best : null,
    depth,
    prune,
    nodes,
    cutoffs,
    timeMs: now() - t0,
    timedOut,
    rootMoves: moves.length,
    completedRootMoves: completed,
  };
}
```

Penjelasan bagian penting

- **Baris awal:** `depth` minimal 1, `prune` default ON, batas waktu default 4000 ms. `cloneState` membuat salinan supaya posisi di layar tidak berubah selama AI mencoba langkah.
- **`aiSide = s.turn`:** siapa pun yang sedang jalan menjadi MAX. Jadi fungsi yang sama juga bisa dipakai untuk menguji AI melawan AI.
- **`nodes` dan `cutoffs`:** penghitung untuk panel statistik dan pengujian. Setiap panggilan `ab()` = satu node.
- **Skor menang `WIN - ply`:** `WIN` = 1.000.000. Dikurangi `ply` (jarak dari akar) supaya menang dalam 2 giliran dinilai lebih tinggi daripada menang dalam 4 giliran. AI jadi memilih kemenangan tercepat, dan kalau kalah, menunda kekalahan selama mungkin.
- **Akar ditangani terpisah** (bukan lewat `ab()`) karena di akar kita perlu tahu *langkah mana* yang terbaik, bukan hanya nilainya. `bestMove` diperbarui setiap ada nilai lebih tinggi.
- **Hasil:** objek berisi `move`, `score`, `depth`, `prune`, `nodes`, `cutoffs`, `timeMs`, `timedOut`. Persis sesuai *Expected Output* di proposal.

7 Alpha-Beta Pruning

Contoh yang sama, dengan pemangkasan



Setelah cabang A, MAX sudah dijamin 3 (alpha = 3). Di B, balasan pertama bernilai 2, jadi MIN pasti memberi ≤ 2. Karena 2 ≤ 3, MAX tidak akan memilih B, dan daun 9 tidak perlu dilihat.

Langkah	Simpul	α	β	Yang terjadi
1	A, daun 3	−∞	3	MIN: best = 3, beta = 3
2	A, daun 5	−∞	3	5 tidak lebih kecil dari 3, A = 3
3	Akar	3	+∞	MAX: best = 3, alpha = 3
4	B, daun 2	3	2	beta = 2 ≤ alpha = 3 → **cutoff**, daun 9 dilewati
5	Akar	3	+∞	B bernilai ≤ 2, A tetap terbaik → pilih A

Kalimat kunci untuk presentasi: "Alpha-Beta tidak menebak. Cabang yang dibuang adalah cabang yang *terbukti* tidak bisa mengubah keputusan, jadi hasilnya selalu sama dengan Minimax biasa. Kami membuktikannya di 30 posisi pada depth 1 sampai 3."

Kode bagian MAX dan MIN

js/ai.js : baris 217–254

```
217   function ab(d, alpha, beta, ply) {
218     nodes++;
219     if ((nodes & 255) === 0 && now() > deadline) throw ABORT;
220     const side = s.turn;
221     if (!hasJump(s, side)) return side === aiSide ? -(WIN - ply) : WIN - ply;
222     if (d === 0) return evaluate(s, aiSide);
223     const moves = generateMoves(s, side);
224     const isMax = side === aiSide;
225     if (d >= CFG.orderMinDepth) orderMoves(s, moves, aiSide, isMax);
226     if (isMax) {
227       let best = -Infinity;
228       for (const m of moves) {
229         const from = make(s, m);
230         const v = ab(d - 1, alpha, beta, ply + 1);
231         unmake(s, m, from);
232         if (v > best) best = v;
233         if (best > alpha) alpha = best;
234         if (prune && beta <= alpha) {
235           cutoffs++;
236           break;
237         }
238       }
239       return best;
240     }
241     let best = Infinity;
242     for (const m of moves) {
243       const from = make(s, m);
244       const v = ab(d - 1, alpha, beta, ply + 1);
245       unmake(s, m, from);
246       if (v < best) best = v;
247       if (best < beta) beta = best;
248       if (prune && beta <= alpha) {
249         cutoffs++;
250         break;
251       }
252     }
253     return best;
254   }
```

Perhatikan `if (prune && …)`: kalau tombol Alpha-Beta OFF, `prune` bernilai `false` dan loop tidak pernah berhenti lebih awal, jadi kode yang sama berubah menjadi Minimax biasa. Itulah kenapa perbandingannya adil: **satu-satunya** perbedaan adalah pemangkasan.

Di akar, alpha juga diperbarui (`if (prune && best > alpha) alpha = best;`) lalu diteruskan ke setiap anak, sedangkan beta di akar selalu `Infinity` karena tidak ada simpul MIN di atas akar.

8 Langkah, make/unmake, dan Kandidat Jebakan

Satu langkah dikemas dalam satu angka

js/ai.js : baris 142–144

```
142  // A move is packed as jump | (trap + 1) << 6, so trap -1 (no square left) packs to 0.
143  export const pack = (jump, trap) => jump | ((trap + 1) << 6);
144  export const unpack = (m) => ({ jump: m & 63, trap: (m >> 6) - 1 });
```

Kotak lompatan 0–48 muat dalam 6 bit (maks. 63). Kotak jebakan + 1 disimpan di bit di atasnya, sehingga jebakan −1 ("tidak ada kotak tersisa") menjadi 0. Contoh: lompat ke 30, jebak 12 → `30 | (13 << 6) = 30 + 832 = 862`. Satu angka lebih hemat memori daripada objek `{ jump, trap }`, dan AI membuat ribuan langkah per detik.

js/ai.js : baris 147–162

```
147  export function generateMoves(s, side, out = []) {
148    const { cells, pos } = s;
149    const from = pos[side];
150    const opp = side ^ 1;
151    for (const j of NEIGHBORS[from]) {
152      if (!open(cells, pos, j)) continue;
153      cells[from] = BURNED;
154      pos[side] = j;
155      trapCandidates(s, opp, trapBuf);
156      if (trapBuf.length === 0) out.push(pack(j, -1));
157      else for (const t of trapBuf) out.push(pack(j, t));
158      pos[side] = from;
159      cells[from] = EMPTY;
160    }
161    return out;
162  }
```

Untuk setiap lompatan legal, knight dipindahkan sementara (kotak asal dibakar) supaya kandidat jebakan dihitung dari posisi *setelah* lompatan, lalu semuanya dikembalikan. Ini persis aturan: jebak dilakukan setelah lompat.

make / unmake: coba lalu batalkan tanpa menyalin papan

js/ai.js : baris 164–181

```
164 function make(s, m) {
165   const side = s.turn;
166   const from = s.pos[side];
167   const t = (m >> 6) - 1;
168   s.cells[from] = BURNED;
169   s.pos[side] = m & 63;
170   if (t >= 0) s.cells[t] = TRAP;
171   s.turn ^= 1;
172   return from;
173 }
174
175 function unmake(s, m, from) {
176   s.turn ^= 1;
177   const t = (m >> 6) - 1;
178   if (t >= 0) s.cells[t] = EMPTY;
179   s.pos[s.turn] = from;
180   s.cells[from] = EMPTY;
181 }
```

Proposal: "langkah dicoba dan dibatalkan langsung pada papan yang sama tanpa menyalin papan, supaya cepat". `make` mengembalikan `from` (kotak asal) karena itu satu-satunya informasi yang hilang dan dibutuhkan `unmake`. Urutan di `unmake` adalah kebalikan `make`.

9 Tiga Pembatasan Agar Cepat

1. Kandidat jebakan maksimal 12

js/ai.js : baris 112–140

```
112 function trapCandidates(s, opp, out) {
113   const { cells, pos } = s;
114   const max = CFG.maxTrapCandidates;
115   out.length = 0;
116   stampId++;
117   const ring1 = NEIGHBORS[pos[opp]];
118   for (const n of ring1) {
119     if (open(cells, pos, n)) {
120       stamp[n] = stampId;
121       out.push(n);
122     }
123   }
124   for (const n of ring1) {
125     if (out.length >= max) break;
126     if (!open(cells, pos, n)) continue;
127     for (const m of NEIGHBORS[n]) {
128       if (out.length >= max) break;
129       if (stamp[m] !== stampId && open(cells, pos, m)) {
130         stamp[m] = stampId;
131         out.push(m);
132       }
133     }
134   }
135   // The opponent is boxed in already: any open square is a legal trap.
136   if (out.length === 0) {
137     for (let i = 0; i < SIZE && out.length < max; i++) if (open(cells, pos, i)) out.push(i);
138   }
139   return out;
140 }
```

Tanpa batas, satu lompatan bisa diikuti ±45 pilihan jebakan. Jebakan yang paling berguna biasanya di dekat lawan, jadi urutannya: (1) kotak tujuan lompatan lawan (ring 1), (2) kotak dua lompatan dari lawan (ring 2), berhenti di 12. `stamp` / `stampId` adalah trik untuk menandai "sudah dimasukkan" tanpa mengosongkan array setiap kali: cukup naikkan `stampId`. Kalau lawan sudah terkurung (tidak ada kotak di ring 1/2), kotak terbuka apa pun dipakai.

2. Pengurutan langkah (move ordering)

js/ai.js : baris 189–198

```
189 function orderMoves(s, moves, aiSide, maximizing) {
190   const scored = moves.map((m) => {
191     const from = make(s, m);
192     const v = evaluate(s, aiSide);
193     unmake(s, m, from);
194     return { m, v };
195   });
196   scored.sort(maximizing ? (a, b) => b.v - a.v : (a, b) => a.v - b.v);
197   for (let i = 0; i < moves.length; i++) moves[i] = scored[i].m;
198 }
```

Alpha-Beta memangkas paling banyak kalau langkah terbaik diperiksa **lebih dulu**. Setiap langkah dinilai sekali dengan `evaluate()` (evaluasi statis), lalu diurutkan: MAX dari besar ke kecil, MIN dari kecil ke besar. Pengurutan sendiri butuh waktu, jadi hanya dilakukan kalau sisa kedalaman ≥ 2 (`orderMinDepth`), sesuai proposal.

3. Batas waktu 4 detik

`js/ai.js : baris 217–219`

```
217    function ab(d, alpha, beta, ply) {
218      nodes++;
219      if ((nodes & 255) === 0 && now() > deadline) throw ABORT;
```

Setiap 256 node (`(nodes & 255) === 0`, lebih murah daripada mengecek jam di setiap node), waktu dicek. Kalau melewati batas, dilempar `ABORT`. Di akar, `try/catch` menangkapnya dan AI memakai `bestMove` dari langkah akar yang sudah selesai dihitung (`completed`). `timedOut: true` lalu muncul sebagai label "batas 4 dtk" di panel. Pada hasil uji, Expert maksimum 626 ms, jadi batas ini adalah pengaman, bukan sesuatu yang sering terjadi.

10 Tingkat Kesulitan, Tombol Alpha-Beta, Panel Statistik

js/config.js : baris 10–24

```
10  export const DIFFICULTY = Object.freeze({
11    easy: 1,
12    medium: 2,
13    hard: 3,
14    expert: 4,
15  });
16
17  export const AI = Object.freeze({
18    timeLimitMs: 4000,
19    maxTrapCandidates: 12,
20    orderMinDepth: 2,
21    // Shortest visible "thinking" time so instant replies at Easy do not feel jarring.
22    // UI only; not counted in the reported think time.
23    minUiDelayMs: 450,
24  });
```

Level hanya mengubah **kedalaman**. Easy = 1 (AI hanya melihat langkahnya sendiri), Expert = 4 (AI, manusia, AI, manusia).

js/main.js : baris 501–511

```
501  function startAiGame() {
502    const humanFirst = settings.first === 'human' || (settings.first === 'random' && Math.random() <
0.5);
503    const hs = humanFirst ? 0 : 1;
504    const players = [];
505    players[hs] = { kind: 'human', name: t('you'), sub: colorName(hs) };
506    players[hs ^ 1] = { kind: 'ai', name: `AI · ${t(settings.diff)}`, sub: colorName(hs ^ 1) };
507    $('#game-ab').checked = settings.prune;
508    $('#ab-state').textContent = settings.prune ? 'ON' : 'OFF';
509    // The game keeps the point values it started with, even if the admin changes them mid-game.
510    launch({ mode: 'ai', level: settings.diff, players, rules, ai: { depth: DIFFICULTY[settings.diff],
prune: settings.prune } });
511  }
```

js/main.js : baris 392–412

```
392    stats(res) {
393      const set = (id, html) => {
394        const el = $(id);
395        el.innerHTML = html;
396        el.classList.remove('flash');
397        void el.offsetWidth;
398        el.classList.add('flash');
399      };
400      set('#st-depth', String(res.depth));
401      set('#st-nodes', res.nodes.toLocaleString());
402      set('#st-cutoffs', res.prune ? res.cutoffs.toLocaleString() : '0 <span class="tag">OFF</span>');
403      const ms = res.timeMs < 10 ? res.timeMs.toFixed(1) : Math.round(res.timeMs).toLocaleString();
404      set('#st-time', `${ms} ms${res.timedOut ? ` <span class="tag">${t('timedOut')}</span>` : ''}`);
405      let score = '–';
406      if (res.score != null) {
407        if (res.score >= WIN / 2) score = t('scoreWin', WIN - res.score);
408        else if (res.score <= -WIN / 2) score = t('scoreLoss', WIN + res.score);
409        else score = (res.score > 0 ? '+' : '') + (Number.isInteger(res.score) ? res.score :
res.score.toFixed(1));
410      }
411      set('#st-score', score);
412    },
```

Panel	Isi	Dari
Kedalaman	Depth pencarian	`res.depth`
Posisi diperiksa	Jumlah node	`res.nodes`
Cabang dipangkas	Jumlah cutoff (0 + "OFF" kalau Alpha-Beta mati)	`res.cutoffs`
Waktu	Lama berpikir (+ "batas 4 dtk" kalau habis waktu)	`res.timeMs`, `res.timedOut`
Skor langkah	Nilai evaluasi, atau "Menang/Kalah dalam n giliran" kalau skor ≥ WIN/2	`res.score`

> **Demo yang meyakinkan:** main di Hard, catat "Posisi diperiksa". Matikan tombol Alpha-Beta di panel, main satu langkah lagi: angka posisi diperiksa melonjak dan waktunya naik, tetapi gaya bermain AI tidak berubah.

11 Pengujian Kebenaran Alpha-Beta

tests/run.mjs : baris 111–146

```
111  // ---------------------------------------------------------------------------
112  console.log('\n2. Alpha-Beta correctness (Alpha-Beta vs plain Minimax, same eval/order, no time
limit)');
113  const positions = [];
114  {
115    let seed = BASE_SEED;
116    while (positions.length < 30) {
117      const rng = mulberry32(seed);
118      const turns = 4 + Math.floor(rng() * 5);
119      const s = createState();
120      let ok = true;
121      for (let i = 0; i < turns; i++) {
122        const m = randomMove(s, rng);
123        if (!m) { ok = false; break; }
124        applyMove(s, m);
125      }
126      if (ok && winner(s) === null) positions.push({ seed, turns, s });
127      seed++;
128    }
129    console.log(`   30 positions from seeds ${positions[0].seed}..${positions.at(-1).seed} (seed
${BASE_SEED} + k, random play of 4-8 turns)`);
130    console.log('   depth | identical score | identical move | nodes Minimax | nodes Alpha-Beta | saved |
cutoffs | ms MM | ms AB');
131    for (const depth of [1, 2, 3]) {
132      let sameScore = 0, sameMove = 0, nMM = 0, nAB = 0, cut = 0, tMM = 0, tAB = 0;
133      for (const p of positions) {
134        const mm = search(p.s, { depth, prune: false, timeLimitMs: Infinity, now:
performance.now.bind(performance) });
135        const ab = search(p.s, { depth, prune: true, timeLimitMs: Infinity, now:
performance.now.bind(performance) });
136        if (mm.score === ab.score) sameScore++;
137        else console.log(`   mismatch seed ${p.seed} depth ${depth}: MM ${mm.score} AB ${ab.score}`);
138        if (mm.move.jump === ab.move.jump && mm.move.trap === ab.move.trap) sameMove++;
139        nMM += mm.nodes; nAB += ab.nodes; cut += ab.cutoffs; tMM += mm.timeMs; tAB += ab.timeMs;
140      }
141      const saved = (100 * (1 - nAB / nMM)).toFixed(1) + '%';
142      console.log(`   ${pad(depth, 5)} | ${pad(sameScore + '/30', 15)} | ${pad(sameMove + '/30', 13)} |
${pad(nMM, 13)} | ${pad(nAB, 16)} | ${pad(saved, 5)} | ${pad(cut, 7)} | ${pad(tMM.toFixed(0), 5)} |
${pad(tAB.toFixed(0), 5)}`);
143      check(`depth ${depth}: identical scores`, sameScore === 30);
144      check(`depth ${depth}: identical moves`, sameMove === 30);
145    }
146  }
```

- **Posisi uji bisa diulang:** `mulberry32(seed)` adalah generator angka acak dengan seed, jadi seed yang sama selalu menghasilkan 30 posisi yang sama.
- **Adil:** keduanya memakai `search()` yang sama dengan `timeLimitMs: Infinity`; hanya `prune` yang berbeda.
- **Lulus** kalau skor dan langkah identik di 30/30 posisi pada setiap depth.

12 Hasil Pengujian (Proposal Bab 2.2)

Semua angka di bawah berasal dari `node tests/run.mjs` (run penuh: 100 game per level per lawan). Hasil akhir run: **927 cek lulus, 0 gagal**. Jalankan sendiri sebelum presentasi supaya kamu tahu prosesnya: `node tests/run.mjs --quick` (±30 detik) atau tanpa `--quick` (±5 menit).

1. Kebenaran Alpha-Beta (Tabel 2.2.3, baris 1)

30 posisi uji dibuat dari permainan acak 4–8 giliran dengan seed `20261028 + k` (bisa diulang siapa pun). Alpha-Beta dan Minimax biasa memakai evaluasi, urutan langkah, dan tanpa batas waktu yang sama; bedanya hanya pemangkasan.

Depth	Skor identik	Langkah identik	Node Minimax	Node Alpha-Beta	Hemat	Cutoff	ms Minimax	ms Alpha-Beta
1	30/30	30/30	872	872	0,0%	0	10	6
2	30/30	30/30	24.540	7.050	71,3%	781	91	26
3	30/30	30/30	739.226	37.689	94,9%	2.242	2.063	191

Cara membaca: di depth 3, Alpha-Beta hanya memeriksa sekitar 1 dari 20 posisi yang diperiksa Minimax biasa, ±10× lebih cepat, dan tetap memilih skor dan langkah yang sama di 30 dari 30 posisi. Di depth 1 tidak ada penghematan karena belum ada simpul MIN yang bisa dipangkas (AI hanya melihat langkahnya sendiri).

2. Kekuatan AI / win rate (Tabel 2.2.3, baris 2)

Level	Depth	vs pemain acak	vs pemain greedy
Easy	1	100%	57%
Medium	2	100%	74%
Hard	3	100%	93%
Expert	4	100%	86%

Target proposal "Medium ke atas menang minimal 90% melawan pemain acak" **tercapai** (semua level 100%). Melawan greedy, win rate naik dari Easy ke Hard, tetapi **Expert (86%) sedikit di bawah Hard (93%)**. Jadi target "win rate naik seiring level" hanya tercapai sebagian. Jawaban jujur kalau ditanya:

- Dengan 100 game, selisih 7 poin persen masih dalam rentang acak (pemain greedy memilih acak di antara langkah yang sama bagusnya).
- Fungsi evaluasi BFS/DFS adalah perkiraan. Melihat lebih dalam dengan evaluasi yang tidak sempurna tidak selalu lebih baik (*horizon effect*).
- Pembatasan 12 kandidat jebakan bisa membuang langkah yang ternyata bagus di kedalaman lebih besar.

3. Waktu respons (Tabel 2.2.3, baris 3)

Level	Langkah diukur	Rata-rata	p95	Maksimum
Easy	931	0,1 ms	0,3 ms	0,4 ms
Medium	934	1,5 ms	4,4 ms	12,1 ms
Hard	914	18,5 ms	41,5 ms	82,7 ms
Expert	915	101,1 ms	316,3 ms	626,1 ms

Target "semua level di bawah 4 detik, Easy–Hard di bawah 1 detik" **tercapai jauh**. Angka ini diukur di Node.js dengan kode yang sama dengan Web Worker di browser; di laptop yang lebih lambat angkanya bisa lebih besar, tetapi masih jauh dari 4 detik. *p95* artinya 95% langkah selesai di bawah angka itu.

4. Hal yang belum dibuktikan oleh tes otomatis

- **Playtest** (±10 siswa, kuesioner skala 1–5) adalah uji manusia; hasilnya harus dicatat sendiri oleh tim. Jangan sebut angka playtest yang tidak benar-benar dilakukan.
- Waktu respons di browser bisa dicek langsung di panel **Otak AI** saat bermain (kolom Waktu).

tests/run.mjs : baris 16–22

```
16  function check(name, cond, detail = '') {
17    if (cond) passes++;
18    else {
19      failures++;
20      console.log(`  FAIL ${name}${detail ? ' :: ' + detail : ''}`);
21    }
22  }
```

13 Fitur Rally: Poin, Leaderboard, Admin

Fitur ini **tidak ada di proposal**; ditambahkan supaya game bisa dipakai di rally NPLC. Bukan bagian dari empat algoritma AI, tetapi mungkin ditanya saat demo.

Level	Pengali	Per langkah	Bonus menang
Easy	×1	+10	+200
Medium	×1,5	+15	+300
Hard	×2	+20	+400
Expert	×2,5	+25	+500

Hanya mode Single Player (lawan AI) yang memberi poin. Contoh: menang di Hard dengan 12 langkah = 12 × 20 + 400 = **640 poin**.

js/scoring.js : baris 64–96

```
64  export function scoreGame(moves, { mode = 'ai', level, humanSide, rules = SCORING }) {
65    const mult = rules.multiplier?.[level] ?? 0;
66    const out = {
67      valid: true, badMove: -1, finished: false, won: false, mult, steps: [],
68      humanMoves: 0, movePoints: 0, winPoints: 0, total: 0,
69    };
70    if (!pointsEnabled(mode) || !Object.hasOwn(DIFFICULTY, level) || !Array.isArray(moves)) return out;
71
72    const per = levelValues(rules, level);
73    const s = createState();
74    for (let i = 0; i < moves.length; i++) {
75      const m = moves[i];
76      if (!validateMove(s, m)) {
77        out.valid = false;
78        out.badMove = i;
79        return out;
80      }
81      const side = s.turn;
82      applyJump(s, m.jump);
83      applyTrap(s, m.trap);
84      if (side === humanSide) out.humanMoves++;
85      out.steps.push(side === humanSide ? per.move : 0);
86      endTurn(s);
87    }
88
89    const w = winner(s);
90    out.finished = w !== null;
91    out.won = w === humanSide;
92    out.movePoints = out.humanMoves * per.move;
93    if (out.won) out.winPoints = per.win;
94    out.total = out.movePoints + out.winPoints;
95    return out;
96  }
```

- **Anti curang:** browser hanya mengirim daftar langkah. Server (`api/score.js`) memutar ulang semua langkah dengan `rules.js` yang sama, menolak langkah ilegal atau game yang belum selesai, lalu menghitung poin sendiri.

- **Leaderboard:** total poin per tim dari semua game, urut total → jumlah menang → yang lebih dulu mengirim. Disimpan di Upstash Redis lewat Vercel.
- **Admin:** `/admin.html` dengan kunci `ADMIN_KEY` bisa mengubah poin per langkah, bonus menang, dan pengali tiap level.

14 Latihan

Latihan 1: Alpha-Beta dengan tangan

MAX punya tiga langkah. Daun (kiri ke kanan) di bawah tiap simpul MIN: A = [4, 8], B = [6, 1, 7], C = [3, 9]. Hitung nilai akar dan daun mana yang dipangkas.

Jawaban: A = 4 (α = 4). B: daun 6 → β = 6, daun 1 → β = 1 ≤ 4 → cutoff, daun 7 dilewati, B ≤ 1. C: daun 3 → β = 3 ≤ 4 → cutoff, daun 9 dilewati. Akar = 4, pilih A. Dua daun dipangkas.

Latihan 2: Packing

Berapa `pack(6, -1)` dan apa hasil `unpack(1999)`?

Jawaban: `6 | (0 << 6) = 6`. 1999 = 31 × 64 + 15 → jump = 15, trap = 31 − 1 = 30.

Latihan 3: Coba di kode

1. Ubah `maxTrapCandidates` dari 12 ke 6 di `config.js`, jalankan `node tests/run.mjs --quick`. Apa yang terjadi pada waktu dan win rate? (Kembalikan setelahnya.)
2. Ubah `orderMinDepth` ke 99 (praktis mematikan pengurutan). Bandingkan kolom "nodes Alpha-Beta" di bagian 2.

15 Pertanyaan Presentasi yang Mungkin Muncul

T: Apa bedanya Minimax dan Alpha-Beta?

J: Minimax memeriksa semua cabang. Alpha-Beta memakai batas α dan β untuk melewati cabang yang terbukti tidak mengubah keputusan. Hasilnya sama, node jauh lebih sedikit: di depth 3 hemat 94,9%.

T: Bagaimana kalian yakin hasilnya sama?

J: Uji di 30 posisi dari seed tetap, depth 1–3: skor dan langkah identik 30/30. Bedanya hanya flag `prune` di fungsi yang sama.

T: Kenapa depth 1 tidak ada penghematan?

J: Di depth 1 semua anak akar langsung dievaluasi; tidak ada simpul MIN tempat β bisa turun di bawah α, jadi tidak ada cutoff.

T: Kenapa kandidat jebakan dibatasi 12? Bukankah itu bisa membuang langkah bagus?

J: Bisa, dan itu trade-off yang kami pilih. Tanpa batas, branching factor bisa ±360 dan Expert tidak akan selesai dalam 4 detik. Jebakan yang efektif hampir selalu di sekitar lawan, jadi ring 1 dan ring 2 dipilih lebih dulu.

T: Apa yang terjadi kalau waktu habis?

J: Pencarian dihentikan dengan exception `ABORT`; AI memakai langkah terbaik dari langkah akar yang sudah selesai dihitung, dan panel menampilkan "batas 4 dtk".

T: Kenapa skor menang dikurangi ply?

J: Supaya AI memilih kemenangan tercepat dan, kalau kalah, kekalahan terlambat. Tanpa itu, semua kemenangan bernilai sama dan AI bisa "bermain-main".

T: Kenapa Expert kalah win rate dari Hard melawan greedy?

J: Lihat Bab Hasil Pengujian: variasi 100 game, evaluasi BFS/DFS yang berupa perkiraan, dan pembatasan kandidat. Melawan pemain acak semua level tetap 100%.

T: Apakah ini memakai machine learning seperti AlphaZero [2]?

J: Tidak. Referensi [2] dipakai sebagai konteks game AI. Kami hanya memakai empat algoritma yang diizinkan: Minimax, Alpha-Beta, BFS, DFS. Tidak ada training.

16 Checklist Sebelum Presentasi

- ☐ Bisa menggambar contoh pohon Alpha-Beta di papan tulis dan menunjuk cutoff-nya.
- ☐ Bisa membuka `js/ai.js` dan menunjuk baris untuk setiap baris pseudocode.
- ☐ Hafal angka: depth 3 → 739.226 vs 37.689 node (hemat 94,9%), 30/30 identik.
- ☐ Sudah mencoba demo Alpha-Beta ON/OFF di Hard dan melihat panel berubah.
- ☐ Sudah menjalankan `node tests/run.mjs --quick` sendiri.
- ☐ Bisa menjelaskan alur satu giliran (Bab 4) tanpa melihat catatan.